

```
In [34]: import pandas as pd
```

```
In [35]: df = pd.read_csv(r"E:\pradnya's programming\Domains\ML\DA_clgAss\fortune500.csv")
```

```
In [36]: df.head()
```

```
Out[36]:
```

	Year	Rank	Company	Revenue (in millions)	Profit (in millions)
0	1955	1	General Motors	9823.5	806
1	1955	2	Exxon Mobil	5661.4	584.8
2	1955	3	U.S. Steel	3250.4	195.4
3	1955	4	General Electric	2959.1	212.6
4	1955	5	Esmark	2510.8	19.1

```
In [37]: print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29500 entries, 0 to 29499
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Year                  29500 non-null  int64
1   Rank                  29500 non-null  int64
2   Company               29500 non-null  object
3   Revenue (in millions) 29500 non-null  object
4   Profit (in millions)  29500 non-null  object
dtypes: int64(2), object(3)
memory usage: 1.1+ MB
None
```

```
In [38]: df.shape
```

```
Out[38]: (29500, 5)
```

```
In [39]: # isnull() returns a DataFrame of Boolean values, where True represents missing data (NaN). This is useful when you want to Lo
null_values = df.isnull()
print(null_values)
```

	Year	Rank	Company	Revenue (in millions)	Profit (in millions)
0	False	False	False	False	False
1	False	False	False	False	False
2	False	False	False	False	False
3	False	False	False	False	False
4	False	False	False	False	False
...	...	...	...	...	...
29495	False	False	False	False	False
29496	False	False	False	False	False
29497	False	False	False	False	False
29498	False	False	False	False	False
29499	False	False	False	False	False

[29500 rows x 5 columns]

```
In [40]: print(df.isnull().sum())
```

```
Year          0
Rank          0
Company       0
Revenue (in millions)  0
Profit (in millions)  0
dtype: int64
```

```
In [41]: df.describe()
```

Out[41]:

	Year	Rank
count	29500.000000	29500.000000
mean	1983.728814	284.397966
std	16.616417	191.401503
min	1955.000000	1.000000
25%	1969.000000	135.000000
50%	1984.000000	269.000000
75%	1999.000000	403.000000
max	2009.000000	1000.000000

In [42]: `df.describe(include="all")`

Out[42]:

	Year	Rank	Company	Revenue (in millions)	Profit (in millions)
count	29500.000000	29500.000000	29500	29500	29500
unique	NaN	NaN	2588	21924	8677
top	NaN	NaN	CBS	86.2	N.A.
freq	NaN	NaN	61	10	369
mean	1983.728814	284.397966	NaN	NaN	NaN
std	16.616417	191.401503	NaN	NaN	NaN
min	1955.000000	1.000000	NaN	NaN	NaN
25%	1969.000000	135.000000	NaN	NaN	NaN
50%	1984.000000	269.000000	NaN	NaN	NaN
75%	1999.000000	403.000000	NaN	NaN	NaN
max	2009.000000	1000.000000	NaN	NaN	NaN

```
In [43]: # Revenue_mean = df["Revenue (in millions)"].mean()
# Revenue_mean

#error
```

```
In [44]: df['Revenue (in millions)'].tail()
```

```
Out[44]: 29495    1,729.00
29496    1,724.40
29497    1,723.60
29498    1,721.50
29499    1,720.80
Name: Revenue (in millions), dtype: object
```

```
In [45]: # Use the `replace()` function to remove symbols
df['Revenue (in millions)'] = df['Revenue (in millions)'].replace(',', '', regex=True)
df["Revenue (in millions)"].tail()
```

```
Out[45]: 29495    1729.00
          29496    1724.40
          29497    1723.60
          29498    1721.50
          29499    1720.80
          Name: Revenue (in millions), dtype: object
```

```
In [46]: df['Revenue (in millions)'] = df['Revenue (in millions)'].astype(str).astype(float)
```

```
In [47]: x = df["Revenue (in millions)"].mean()
          print(x)
```

```
5237.162101694916
```

```
In [48]: # import the necessary libraries
          import seaborn as sns
          # set the background style of the plot
          sns.set_style('whitegrid')
          sns.distplot(df['Revenue (in millions)'], kde = False, color = 'red', bins = 30)
```

C:\Users\Pradnya\AppData\Local\Temp\ipykernel_15332\3380783093.py:5: UserWarning:

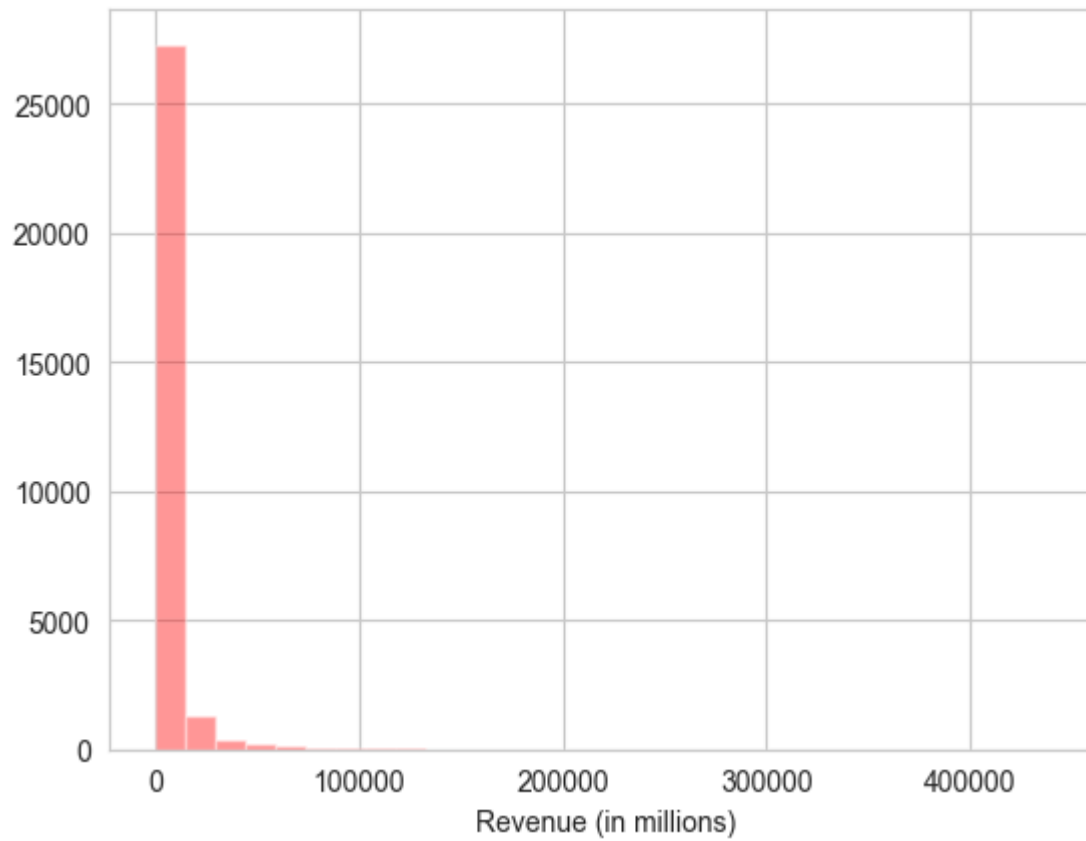
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(df['Revenue (in millions)'], kde = False, color = 'red', bins = 30)
```

```
Out[48]: <Axes: xlabel='Revenue (in millions)'>
```



```
In [49]: (df["Revenue (in millions)"].mean())
```

```
Out[49]: np.float64(5237.162101694916)
```

```
In [50]: df["Revenue (in millions)"].max()
```

```
Out[50]: np.float64(442851.0)
```

```
In [51]: df["Revenue (in millions)"].min()
```

```
Out[51]: np.float64(49.7)
```

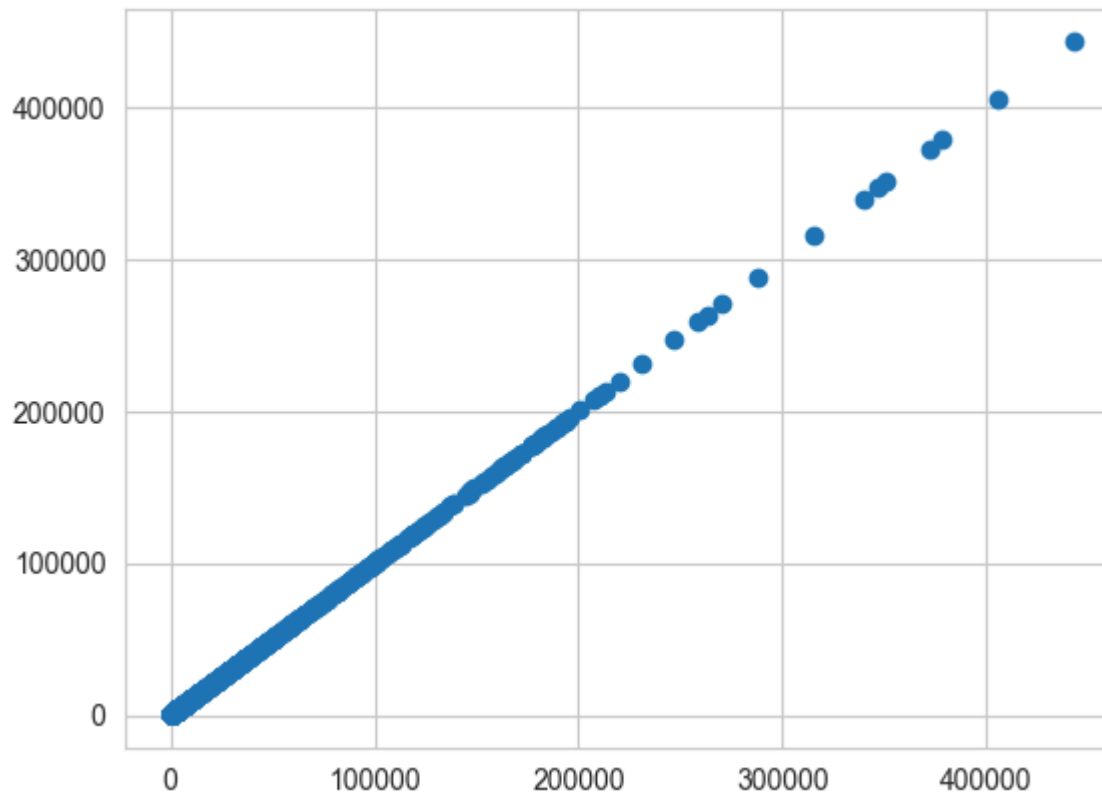
```
In [52]: standrad_deviation = df["Revenue (in millions)"].std()  
standrad_deviation
```

```
Out[52]: np.float64(14335.344472853512)
```

```
In [53]: variance = df["Revenue (in millions)"].var()  
variance
```

```
Out[53]: np.float64(205502101.15537173)
```

```
In [63]: import matplotlib.pyplot as plt  
# st1 = [df['Year'].std(),df['Revenue (in millions)'].std()]  
# var1 = [df['Year'].var(),df['Revenue (in millions)'].var()]  
# plt.scatter(st1, var1)  
# plt.show()  
  
st1 = [df['Year'],df['Revenue (in millions)']]  
var1 = [df['Year'],df['Revenue (in millions)']]  
plt.scatter(st1, var1)  
plt.show()
```

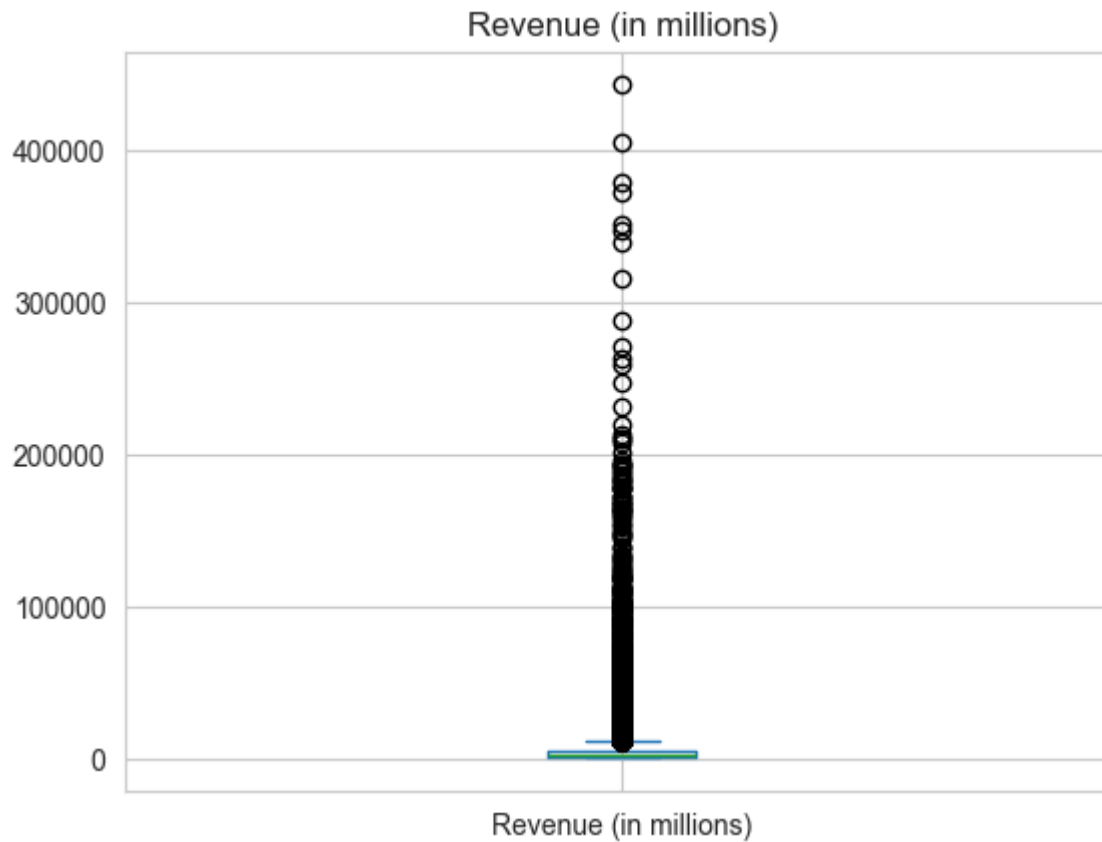


```
In [64]: Q2 = df ['Revenue (in millions)'].quantile(0.5)
Q1 = df ['Revenue (in millions)'].quantile(0.25)
Q3 = df ['Revenue (in millions)'].quantile(0.75)
print(f"Q1:{Q1}\n Q2:{Q2}\n Q3:{Q3}\n")
```

```
Q1:443.275
Q2:1498.0500000000002
Q3:4607.825
```

```
In [65]: import matplotlib.pyplot as plt
```

```
In [66]: df['Revenue (in millions)'].plot(kind='box', title='Revenue (in millions)')
plt.show()
```



```
In [ ]: # see if the data is normally distributed by calculating skewness
# importing SciPy
import scipy.stats as stats
skewness = stats.skew(df['Revenue (in millions)'], axis=0, bias=True)
print(skewness)

# Skewness = 0: Then normally distributed.
# Skewness > 0: Then more weight in the left tail of the distribution.
# Skewness < 0: Then more weight in the right tail of the distribution.
```

10.718020433302923

```
In [75]: kurtosis = stats.kurtosis(df['Revenue (in millions)'], axis = 0, fisher = True, bias = True)
print(kurtosis)
```

```
# kurtosis for normal distribution is equal to 3.  
# For a distribution having kurtosis < 3: It is called platykurtic.  
# For a distribution having kurtosis > 3, It is called leptokurtic and it signifies that it tries to produce more outliers rat
```

186.36068493585537

In []: [# https://www.geeksforgeeks.org/how-to-calculate-skewness-and-kurtosis-in-python/](https://www.geeksforgeeks.org/how-to-calculate-skewness-and-kurtosis-in-python/)